Silver OBT Watermarking

Monday, 16 March 2026 10:43 AM

Uber Project: Streaming Joins, Watermarking, and the Silver OBT

1. Introduction to Streaming Joins in the Silver Layer

The final step in the Silver layer is the creation of the **Silver One Big Table (OBT)**. This involves joining the streaming staging table (`stg_rides`) with various batch mapping tables. To handle this in a real-time environment, we utilize **Streaming Joins** within the Spark Declarative Pipeline (SDP) framework.

2. Managing Late Data with Watermarking

A critical challenge in streaming architectures is handling events that arrive late due to network latency or other issues. To address this, we use the concept of **Watermarking**.

What is a Watermark?

- **Definition**: A watermark is a threshold that tells Spark how long to wait for late-arriving events.
- **State Management**: It is essential for **stateful transformations**. Since Spark cannot remember the state of all events indefinitely, a watermark sets a deadline.
- **Implementation**: In our pipeline, we set a watermark on the `booking_timestamp` with a delay of **5 minutes**.
  - **Example**: If a ride occurs at 10:00 AM but the event arrives at 10:04 AM, it is processed. If it arrives at 10:06 AM, it may be dropped to prevent infinite waiting and resource exhaustion.

3. Implementation: `silver_obt` Streaming Table

The `silver_obt` table is defined as a streaming table that continuously updates as new ride events arrive. It uses the `STREAM` keyword to indicate incremental reading from the source.

3.1. SQL Implementation with DLT

```
CREATE OR REFRESH STREAMING TABLE silver_obt

AS

    SELECT

        stg_rides.*, -- Core ride data

        map_vehicle_makes.vehicle_make,

        map_vehicle_types.vehicle_type, map_vehicle_types.description,
map_vehicle_types.base_rate,

        map_ride_statuses.ride_status,

        map_payment_methods.payment_method, map_payment_methods.is_card,

        map_cities.city as pickup_city, map_cities.state, map_cities.region,

        map_cancellation_reasons.cancellation_reason

    FROM

        STREAM(uber.bronze.stg_rides)

        WATERMARK booking_timestamp DELAY OF INTERVAL 5 MINUTES stg_rides

    LEFT JOIN uber.bronze.map_vehicle_makes map_vehicle_makes

        ON stg_rides.vehicle_make_id = map_vehicle_makes.vehicle_make_id

    LEFT JOIN uber.bronze.map_vehicle_types map_vehicle_types
```

```
    LEFT JOIN uber.bronze.map_vehicle_types map_vehicle_types
        ON stg_rides.vehicle_type_id = map_vehicle_types.vehicle_type_id
    LEFT JOIN uber.bronze.map_ride_statuses map_ride_statuses
        ON stg_rides.ride_status_id = map_ride_statuses.ride_status_id
    LEFT JOIN uber.bronze.map_payment_methods map_payment_methods
        ON stg_rides.payment_method_id = map_payment_methods.payment_method_id
    LEFT JOIN uber.bronze.map_cities map_cities
        ON stg_rides.pickup_city_id = map_cities.city_id
    LEFT JOIN uber.bronze.map_cancellation_reasons map_cancellation_reasons
        ON stg_rides.cancellation_reason_id =
map_cancellation_reasons.cancellation_reason_id
```

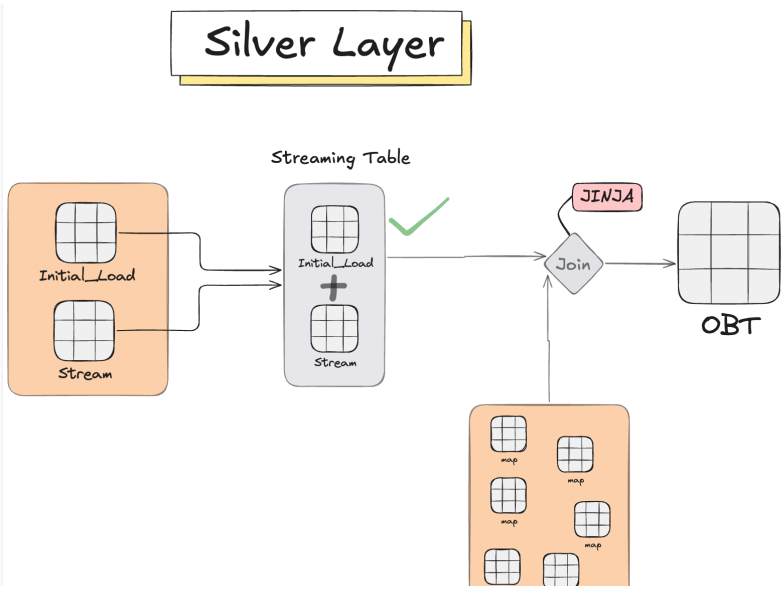
4. Key Components of the OBT Pipeline

- **`CREATE OR REFRESH STREAMING TABLE`**: Ensures the table is either created or updated incrementally with new data.
- **`STREAM(table_name)`**: Directs Spark to read the source table as a continuous stream rather than a static batch.
- **`LEFT JOIN` on Mapping Tables**: Converts technical IDs (e.g., `vehicle_type_id = 2`) into human-readable attributes (e.g., `vehicle_type = SUV`). This "flattens" the data into a wide, descriptive format.

5. The One Big Table (OBT) Concept

The `silver_obt` is the definitive "One Big Table" for the project. By consolidating many small lookup tables into a single enriched dataset, we provide a high-quality source for: * **Business Intelligence (BI)**: Powering dashboards and reports. * **Analytics**: Enabling deep dives into ride patterns and performance. * **Machine Learning**: Providing clean, feature-rich data for model training.

6. Full Pipeline Summary

The end-to-end flow is now complete: 1. **Event Hub**: Receives raw events from the Ride App. 2. **`rides_raw` (Bronze)**: Stores raw JSON events. 3. **`stg_rides` (Silver)**: Stores structured, parsed, and unified (historical + streaming) data. 4. **`silver_obt` (Silver/Gold)**: The final enriched analytics table, joined with all necessary dimensions and protected by watermarking.

With the creation of the `silver_obt`, the Silver layer transformations are successfully completed.